

COMPTE RENDU

Programmation Parallèle — Challenge 1 — Factorisation de Fermat
parallèle avec MPI

4e année Cybersécurité (Ing4-CYB/CCV) - École Supérieure
d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BAKHOUYA

Date : 8 octobre 2026

Rappel du sujet

La méthode de factorisation de Fermat repose sur l'identité

$$N = x^2 - y^2 = (x - y)(x + y),$$

où x et y sont des entiers. On part de $x = \lceil \sqrt{N} \rceil$ et on incrémente x jusqu'à ce que $y^2 = x^2 - N$ soit un carré parfait ; les facteurs sont alors $x - y$ et $x + y$.

Chaque valeur candidate de x pouvant être testée indépendamment, la recherche se parallélise sans communication : parmi T processus, le processus t teste $x = \lceil \sqrt{N} \rceil + t$, puis $x + T$, $x + 2T$, etc. Le sujet demande :

- (a) de compléter le programme MPI `fermat.c` ;
- (b) d'évaluer ses performances sur 10 entiers $N = p \times q$ de taille croissante (de 10^{12} à 10^{18}), pour différents nombres de processus, et d'analyser l'évolution du temps d'exécution et du speedup en fonction de N et du nombre de processus.

1 Question a : programme MPI

1.1 Principe

Avec MPI, chaque processus est une copie complète du programme, avec sa propre mémoire. Chaque processus connaît son identifiant (**rank**, de 0 à **size**−1) et le nombre total de processus (**size**). La répartition du travail se fait donc uniquement à partir de **rank** et **size**, et la seule communication nécessaire est la mise en commun du résultat final par `MPI_Allreduce`.

1.2 Code complété

```
unsigned long long N = 10000000400000000231ULL;
unsigned long long start = (unsigned long long)ceil(sqrt((double)N));
int maxTests = 1000000;

unsigned long long localX = 0, localY = 0;

for (int i = rank ; i < maxTests ; i += size) {
    unsigned long long x = start + i ;
    unsigned long long y2 = x * x - N;
    if ( isPerfectSquare(y2) && localX == 0) {
        localX = x ;
        localY = (unsigned long long)sqrt((double)y2) ;
        break;
    }
}

// Reduce results: first non-zero value
unsigned long long globalX = 0, globalY = 0;
MPI_Allreduce(&localX, &globalX, 1, MPI_UNSIGNED_LONG_LONG, MPI_MAX,
    MPI_COMM_WORLD);
```

```
MPI_Allreduce(&localY, &globalY, 1, MPI_UNSIGNED_LONG_LONG, MPI_MAX, MPI_COMM_WORLD);
```

1.3 Justification des éléments complétés

Élément	Valeur	Justification
start	ceil(sqrt((double)N))	Comme $x^2 = N + y^2 \geq N$, on a $x \geq \sqrt{N}$: le plus petit x possible est $\lceil \sqrt{N} \rceil$. Le <code>ceil</code> est nécessaire, un simple cast tronquerait 5,91 en 5 au lieu de 6.
maxTests	1000000	Nombre maximal de candidats testés avant d'abandonner (évite une boucle infinie si N est premier). C'est une borne de sécurité choisie par nous, l'énoncé n'en impose pas.
Boucle for	<code>i = rank ; i < maxTests ; i += size</code>	Le processus <code>rank</code> teste les indices $i = \text{rank}, \text{rank} + \text{size}, \text{rank} + 2\text{size}, \dots$: les ensembles de candidats sont disjoints et leur union couvre tous les x . C'est l'équivalent de $x \leftarrow x + T$ de l'algorithme 1.
x	start + i	Candidat courant.
Condition if	<code>isPerfectSquare(y2) && localX == 0</code>	On teste si $y^2 = x^2 - N$ est un carré parfait. La garde <code>localX == 0</code> signifie « rien trouvé jusqu'ici » (elle est redondante avec le <code>break</code> , mais elle occupe le second trou du template).
localX, localY	<code>x, (unsigned long long) sqrt((double)y2)</code>	On mémorise x et y . Comme <code>y2</code> est déjà un carré parfait, sa racine est exactement entière : pas besoin de <code>ceil</code> .
MPI_Allreduce	<code>&localX, &globalX, ..., MPI_MAX</code> puis idem pour Y	Chaque processus fournit sa valeur locale, MPI applique l'opération et <i>tous</i> les processus reçoivent le résultat. Les processus qui n'ont rien trouvé ont 0 : le maximum récupère la valeur du processus gagnant. Deux appels, un par variable.

TABLE 1 – Trous du template et valeurs retenues

Pour $N = p \times q$ avec p et q premiers, la seule décomposition $N = a \cdot b$ non triviale est (p, q) : il n'existe donc qu'un seul couple (x, y) dans la fenêtre de recherche, ce qui justifie l'usage de `MPI_MAX` pour récupérer le résultat. Le programme se compile avec `mpicc fermat.c -o fermat -lm` et se lance avec `mpirun -np P ./fermat`.

2 Question b : évaluation des performances

2.1 Protocole de mesure

Un programme de test (`main.c`) reprend exactement la même logique que `fermat.c` dans une fonction paramétrée par N , et l'applique aux 10 cas du tableau de l'énoncé, avec

$p \in \{1, 2, 4, 8\}$ processus (lancés avec `mpirun -oversubscribe`). Pour chaque couple (N, p) :

- le temps est mesuré avec `MPI_Wtime()` autour de l'appel de recherche (`MPI_Allreduce` compris), après un `MPI_Barrier` pour synchroniser le départ ;
- le temps retenu est celui du processus le plus lent (`MPI_Reduce` avec `MPI_MAX`), car la fin du calcul est conditionnée par lui ;
- la mesure est répétée 3 fois et le **meilleur temps** est conservé, pour limiter l'effet des aléas du système ;
- un contrôle vérifie que $(x - y)(x + y) = N$: les 40 exécutions renvoient les bons facteurs ;
- la fenêtre de recherche vaut `maxTests` = 5×10^7 candidats.

Le speedup est $S(p) = T_1/T_p$ et l'efficacité $E(p) = S(p)/p$. Machine : Dell Inspiron 16, Ubuntu 24.04 [à compléter : modèle de processeur et nombre de cœurs, commande `nproc`].

Choix méthodologique : balayage complet de la fenêtre

Pour les 10 cas de test, p et q sont proches ($q - p \leq 60$), donc la solution est trouvée dès le tout premier candidat ($i = 0$, voir plus bas). Avec le programme du sujet (qui s'arrête au premier succès), le processus de rang 0 trouverait donc la solution immédiatement. Avec 1 seul processus, le temps serait quasi nul et le speedup n'aurait aucun sens. Avec plusieurs processus, les rangs ≥ 1 ne sauraient pas que la solution est trouvée (aucune communication pendant la boucle) et poursuivraient leur balayage jusqu'à `maxTests`.

Pour obtenir une mesure exploitable, le programme de test **ne s'arrête pas au premier succès** : chaque processus balaie sa part de la fenêtre (`maxTests`/ p candidats). La charge totale est ainsi fixe et identique pour tous les N et tous les p , et on mesure le coût réel de la recherche et la qualité de sa répartition. Ce balayage complet est un mode du programme de test, `fermat.c` du sujet reste inchangé.

2.2 Résultats

Test	N	T_1 (ms)	T_2 (ms)	T_4 (ms)	T_8 (ms)	S_2	S_4	S_8
1	$1,00 \times 10^{12}$	188,8	95,2	45,1	28,5	1,98	4,19	6,63
2	$4,00 \times 10^{12}$	189,1	95,0	44,8	22,2	1,99	4,22	8,51
3	$2,50 \times 10^{13}$	185,3	94,4	45,0	22,1	1,96	4,11	8,37
4	$1,00 \times 10^{14}$	188,9	93,6	45,4	21,7	2,02	4,16	8,69
5	$4,00 \times 10^{14}$	188,9	95,4	45,0	22,2	1,98	4,20	8,53
6	$2,50 \times 10^{15}$	190,4	95,4	44,9	21,7	2,00	4,24	8,79
7	$1,00 \times 10^{16}$	191,9	94,7	45,1	21,9	2,03	4,26	8,76
8	$4,00 \times 10^{16}$	190,4	95,3	45,0	21,7	2,00	4,23	8,76
9	$2,50 \times 10^{17}$	189,0	95,1	44,6	21,4	1,99	4,24	8,85
10	$1,00 \times 10^{18}$	188,3	95,0	48,1	21,3	1,98	3,91	8,84

TABLE 2 – Temps d'exécution (meilleur de 3 mesures) et speedup pour les 10 cas de test

Processus p	Temps moyen (ms)	Speedup moyen	Efficacité moyenne
1	189,1	1,00	100,0 %
2	94,9	1,99	99,6 %
4	45,3	4,18	104,4 %
8	22,5	8,47	105,9 %

TABLE 3 – Moyennes sur les 10 cas de test

2.3 Temps d'exécution en fonction de N

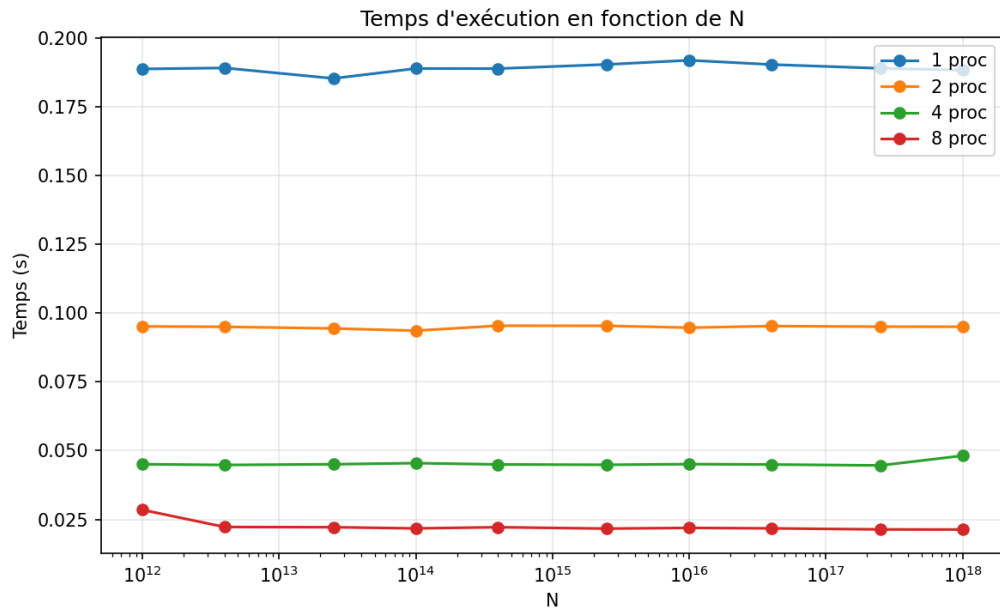


FIGURE 1 – Temps d'exécution en fonction de N (échelle logarithmique en abscisse), une courbe par nombre de processus

Les courbes sont **plates** : pour un nombre de processus donné, le temps ne dépend pas de N . L'écart relatif entre le plus petit et le plus grand temps mesuré est d'environ 3,5 % pour 1 processus, 1,9 % pour 2, 7,8 % pour 4 et 4,3 % pour 8 (hors test 1, voir plus bas), soit du même ordre que le bruit de mesure. En revanche, les courbes sont nettement séparées les unes des autres : c'est le nombre de processus qui détermine le temps.

2.4 Speedup en fonction de N

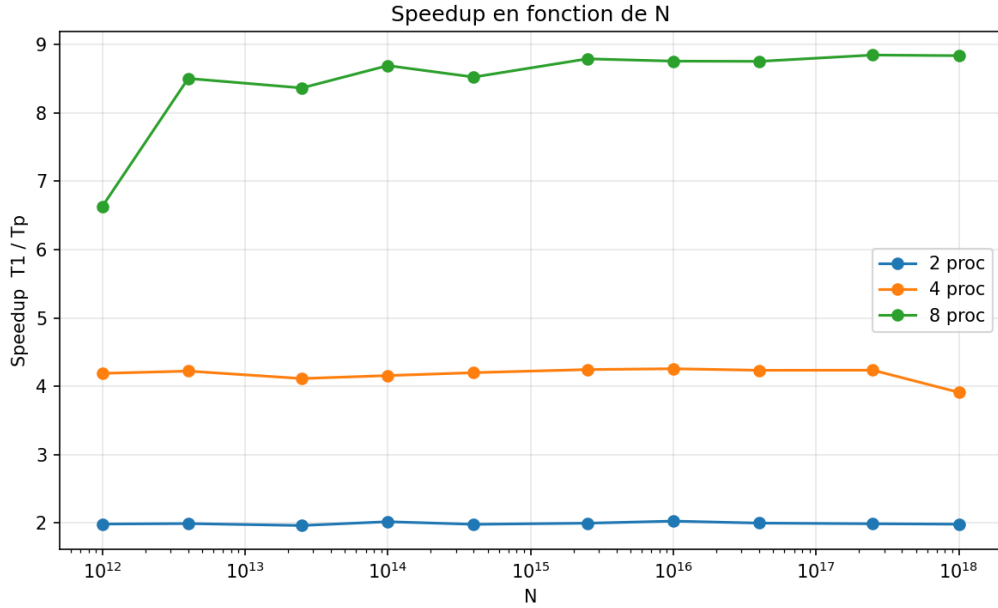


FIGURE 2 – Speedup T_1/T_p en fonction de N (échelle logarithmique en abscisse)

Le speedup est lui aussi **indépendant de N** : il reste voisin de 2 pour 2 processus, de 4,2 pour 4 processus (3,91 pour le test 10) et compris entre 8,4 et 8,9 pour 8 processus. Le seul point qui s'écarte nettement est le test 1 à 8 processus (6,63), discuté ci-dessous.

2.5 Speedup en fonction du nombre de processus

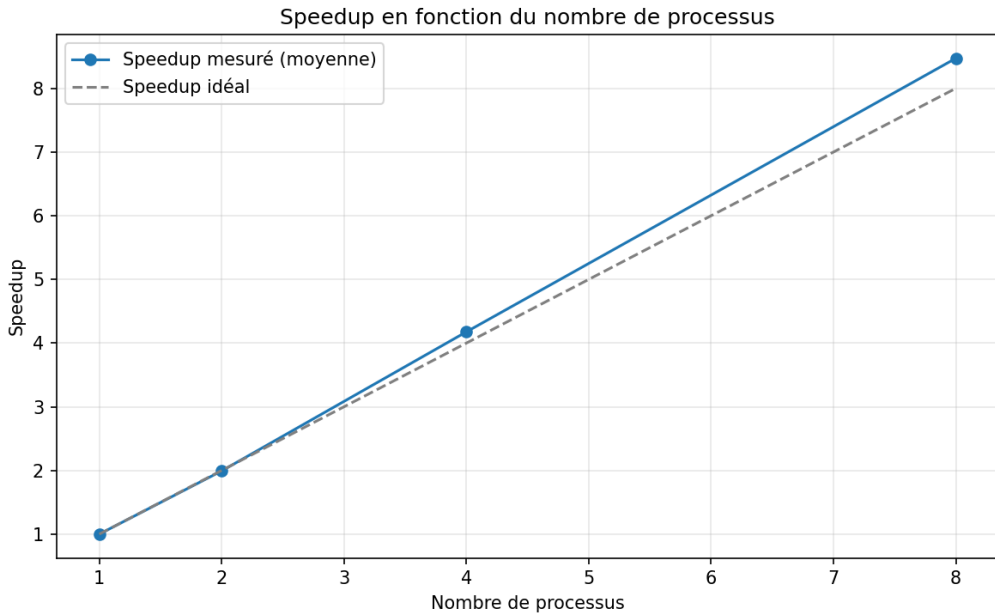


FIGURE 3 – Speedup moyen sur les 10 tests en fonction du nombre de processus, comparé au speedup idéal ($S = p$)

Le speedup moyen suit la droite idéale (1, 1,99, 4,18 et 8,47 pour 1, 2, 4 et 8 processus), avec une efficacité de 99,6 %, 104 % et 106 %.

2.6 Analyse

Influence de la taille de N . Le temps est indépendant de N pour deux raisons.

- **Le coût d’une itération est constant.** Chaque candidat coûte une multiplication, une soustraction, une racine carrée et une comparaison sur 64 bits, quelle que soit la valeur de N (tous les N testés, jusqu’à 10^{18} , tiennent dans un `unsigned long long`). Avec un balayage de 5×10^7 candidats, le coût par candidat est d’environ 3,8 ns pour 1 processus.
- **L’algorithme converge immédiatement sur ces cas.** Le nombre d’itérations nécessaires est lié à l’écart $x - \sqrt{N}$ avec $x = (p + q)/2$:

$$x - \sqrt{N} = \frac{(\sqrt{q} - \sqrt{p})^2}{2} \approx \frac{(q - p)^2}{8\sqrt{N}}.$$

Ici $q - p$ vaut au plus 60 alors que $\sqrt{N} \geq 10^6$: l’écart est très inférieur à 1 et la solution est le premier candidat, $x = \lceil \sqrt{N} \rceil$, pour les 10 cas. Un N plus grand ne rend donc pas la recherche plus longue. Fermat deviendrait coûteux si p et q étaient éloignés l’un de l’autre, puisque l’écart croît comme $(q - p)^2$.

Influence du nombre de processus. Le temps moyen passe de 189,1 ms (1 processus) à 94,9 ms, 45,3 ms et 22,5 ms pour 2, 4 et 8 processus : il est divisé par p , soit un speedup quasi linéaire. C’est le comportement attendu pour trois raisons :

- aucune communication pendant la boucle : les candidats sont indépendants ;
- la seule communication est la paire de `MPI_Allreduce` finale sur une valeur de 8 octets, dont le coût (quelques dizaines de microsecondes au plus) est négligeable devant des temps de plusieurs dizaines de millisecondes ;
- la charge est parfaitement équilibrée : `maxTests` est divisible par p et le coût par candidat est le même pour tous.

Selon la loi d’Amdahl, $S(p) = 1/(f + (1 - f)/p)$ avec f la fraction séquentielle. Ici f est négligeable (initialisation et réduction finale), donc $S(p) \approx p$, ce que confirment les mesures. Ce speedup quasi linéaire est aussi la signature d’un problème *embarrassingly parallel*.

Deux observations particulières.

- **Efficacité légèrement supérieure à 1** pour 4 et 8 processus (104 % et 106 %, soit 4,18 et 8,47). Un speedup surlinéaire est inhabituel. L’algorithme n’en est pas la cause : la boucle ne manipule aucune donnée en mémoire, donc il n’y a pas d’effet de cache, et la charge est simplement divisée. L’écart (environ 4 à 8 %) est du même ordre que la dispersion des mesures entre tests (2 à 8 %). Nous l’attribuons, sans l’avoir vérifié, à l’environnement de mesure (fréquence du processeur, ordonnancement des processus) ; des mesures supplémentaires seraient nécessaires pour trancher.
- **Test 1 à 8 processus** : 28,5 ms contre environ 22 ms pour les 9 autres tests (speedup de 6,63 au lieu de 8,4 à 8,9). C’est le premier test de l’exécution à 8 processus. Nous supposons un effet de démarrage (lancement des processus, montée

en fréquence), mais ce n'est pas démontré. Hors ce point, le speedup moyen à 8 processus monte à 8,68.

Limites de l'expérience.

- Le speedup proche de 8 avec 8 processus suppose que ces 8 processus s'exécutent réellement en parallèle sur la machine.
- Les mesures portent sur une fenêtre de recherche fixe : elles évaluent la qualité de la répartition, pas le temps nécessaire pour trouver les facteurs dans un cas réel où la solution serait lointaine.
- Dans ce cas réel, le programme du sujet n'aurait pas d'arrêt coordonné : un processus qui trouve la solution s'arrête, mais les autres poursuivent leur balayage jusqu'à `maxTests`. On pourrait l'améliorer en testant périodiquement (toutes les K itérations) un indicateur global avec une communication non bloquante (par exemple `MPI_Iallreduce`), pour que tous les processus s'arrêtent peu après la découverte.

Conclusion

- (a) `fermat.c` est complété avec une répartition cyclique des candidats ($i = \text{rank}$, pas `size`) et une mise en commun par `MPI_Allreduce` avec `MPI_MAX`.
- (b) Sur les 10 cas de test, le temps d'exécution est **indépendant de N** (coût par candidat constant, solution trouvée dès le premier candidat) et **inversement proportionnel au nombre de processus** : speedup moyen de 1,99, 4,18 et 8,47 pour 2, 4 et 8 processus, soit une efficacité proche de 100 %.
- Ce passage à l'échelle presque parfait s'explique par l'absence de communication pendant la recherche et par une charge équilibrée : la fraction séquentielle est négligeable (loi d'Amdahl).
- Les mesures ont été faites en balayant toute la fenêtre de recherche, faute de quoi la solution trouvée au premier candidat rendrait la comparaison sans objet. Deux points restent non expliqués par nos données : l'efficacité légèrement supérieure à 1 à 4 et 8 processus et le test 1 à 8 processus.